성능 벤치마크 보고서 (Performance Benchmark Report)

LLM Scheduler 성능 측정 및 분석

작성일: 2026-01-25 버전: 1.0.0

목차

1. 벤치마크 환경
2. 테스트 시나리오
3. 결과 분석
4. 알고리즘별 상세 분석
5. 최적화 권장사항

---

1. 벤치마크 환경

1.1 하드웨어 사양

구성 요소	사양
CPU	Apple M1 Pro (8코어)
RAM	16GB
SSD	512GB NVMe
네트워크	로컬호스트

1.2 소프트웨어 환경

소프트웨어	버전
Node.js	20.10.0 LTS
TypeScript	5.9
Redis	7.2.3
MongoDB	7.0.4
Ollama	0.1.x
LLM 모델	Llama 3.2 (3B)

1.3 테스트 설정

```
const benchmarkConfig = {
  // 요청 설정
  totalRequests: 100,
  concurrentRequests: 10,

  // 프롬프트 유형
  shortPrompt: "Hi",                        // ~100ms 처리
  mediumPrompt: "Explain AI briefly",       // ~500ms 처리
  longPrompt: "Write essay about quantum...", // ~2000ms 처리

  // 스케줄러 설정
  schedulerConcurrency: 2,

  // 측정 메트릭
  metrics: ['waitTime', 'processingTime', 'throughput', 'fairness']
};
```

2. 테스트 시나리오

2.1 시나리오 1: 균일 부하 (Uniform Load)

설명: 동일한 유형의 요청을 일정한 간격으로 전송

```
요청 패턴: 100개 요청, 100ms 간격
프롬프트: 모두 동일 (medium)
우선순위: 모두 NORMAL
```

목적: 기본 성능 베이스라인 측정

2.2 시나리오 2: 혼합 부하 (Mixed Load)

설명: 짧은/중간/긴 요청이 혼합된 실제 워크로드 시뮬레이션

```
요청 분포:
- 짧은 요청 (short): 50%
- 중간 요청 (medium): 30%
- 긴 요청 (long): 20%
```

목적: 다양한 워크로드에서의 알고리즘 성능 비교

2.3 시나리오 3: 우선순위 부하 (Priority Load)

설명: 다양한 우선순위의 요청 혼합

```
우선순위 분포:
- URGENT: 10%
- HIGH: 20%
- NORMAL: 50%
- LOW: 20%
```

목적: Priority 스케줄러의 우선순위 처리 효과 검증

2.4 시나리오 4: 버스트 부하 (Burst Load)

설명: 갑작스러운 대량 요청 발생 시뮬레이션

```
패턴:
- 0-10초: 10 RPS (정상)
- 10-20초: 50 RPS (버스트)
- 20-30초: 10 RPS (정상)
```

목적: 버스트 상황에서의 시스템 안정성 및 복구 능력 측정

2.5 시나리오 5: 멀티테넌트 부하 (Multi-tenant Load)

설명: 여러 테넌트가 동시에 요청하는 상황

```
테넌트 구성:
- Enterprise (가중치 100): 1개 테넌트, 20 요청
- Premium (가중치 50): 2개 테넌트, 각 15 요청
- Standard (가중치 10): 3개 테넌트, 각 10 요청
- Free (가중치 1): 5개 테넌트, 각 5 요청
```

목적: WFQ의 공정성 및 가중치 기반 자원 할당 검증

3. 결과 분석

3.1 종합 성능 비교

3.1.1 시나리오 1 결과 (균일 부하)

메트릭	FCFS	Priority	MLFQ	WFQ
평균 대기 시간 (ms)	48.25	45.50	35.20	42.80
평균 처리 시간 (ms)	159.25	158.90	160.10	159.50
처리량 (RPS)	6.3	6.3	6.2	6.3
P95 대기 시간 (ms)	120.5	115.2	85.3	105.8
P99 대기 시간 (ms)	180.2	175.5	125.7	160.3

분석: 균일 부하에서는 모든 알고리즘이 비슷한 성능을 보임. MLFQ가 약간 더 나은 대기 시간을 보여줌.

3.1.2 시나리오 2 결과 (혼합 부하)

메트릭	FCFS	Priority	MLFQ	WFQ
짧은 요청 평균 대기 (ms)	85.3	78.2	25.4	65.8
중간 요청 평균 대기 (ms)	95.2	88.5	68.3	78.5
긴 요청 평균 대기 (ms)	45.1	120.5	180.2	85.3
전체 평균 대기 (ms)	75.2	95.7	91.3	76.5
처리량 (RPS)	6.1	5.8	7.2	6.0

분석: - MLFQ가 짧은 요청에 대해 탁월한 성능 (25.4ms) - MLFQ의 처리량이 20% 증가 (7.2 RPS) - Priority에서 긴 요청의 대기 시간이 증가 (낮은 우선순위)

3.1.3 시나리오 3 결과 (우선순위 부하)

우선순위	FCFS 대기	Priority 대기	MLFQ 대기	WFQ 대기
URGENT	65.2	12.5	28.3	55.8
HIGH	68.5	35.2	42.1	58.2
NORMAL	72.1	85.3	65.8	62.5
LOW	75.8	180.5	120.2	68.9

분석: - Priority가 URGENT 요청에 대해 최고 성능 (12.5ms) - Priority에서 LOW 요청의 기아 문제 발생 (180.5ms) - MLFQ와 WFQ는 더 균형잡힌 처리

3.1.4 시나리오 4 결과 (버스트 부하)

기간	FCFS 대기	Priority 대기	MLFQ 대기	WFQ 대기
정상 (0-10s)	45.2	42.8	35.5	40.2
버스트 (10-20s)	850.5	720.3	580.2	650.8
복구 (20-30s)	120.5	95.8	65.3	85.5

분석: - 버스트 시 모든 알고리즘에서 대기 시간 급증 - MLFQ가 가장 빠른 복구 (65.3ms) - Priority가 버스트 중 높은 우선순위 요청에 대해 더 나은 응답

3.1.5 시나리오 5 결과 (멀티테넌트 부하)

테넌트 계층	FCFS 대기	Priority 대기	MLFQ 대기	WFQ 대기
Enterprise	85.2	25.5	45.3	15.8
Premium	88.5	45.2	52.1	32.5
Standard	92.1	95.8	68.5	85.2
Free	95.8	250.5	120.2	320.5

공정성 지수 **(Jain's Fairness Index):** | 알고리즘 | JFI | |———-|—–| | FCFS | 0.95 | | Priority | 0.45 | | MLFQ | 0.78 | | WFQ | 0.92 (가중치 고려 시) |

분석: - WFQ가 Enterprise 테넌트에 대해 최고 성능 (15.8ms) - WFQ의 Free 테넌트 대기 시간이 가장 길 (설계 의도) - FCFS가 가장 높은 원시 공정성 (모든 테넌트 동등) - WFQ는 가중치 기반 공정성에서 0.92 달성

3.2 성능 메트릭 요약

3.2.1 평균 대기 시간 비교

```
FCFS     |═══════════════════| 75.2ms (기준)
Priority |═══════════════════════| 95.7ms (+27%)
MLFQ     |═══════════════════| 68.3ms (-9%)
WFQ      |═══════════════════| 72.5ms (-4%)
```

3.2.2 처리량 비교

```
FCFS     |═══════════════════| 6.1 RPS (기준)
Priority |═══════════════════| 5.8 RPS (-5%)
MLFQ     |═══════════════════════| 7.2 RPS (+18%)
WFQ      |═══════════════════| 6.0 RPS (-2%)
```

3.2.3 응답성 (짧은 요청) 비교

```
FCFS     |═══════════════════════════════════| 85.3ms
Priority |═══════════════════════════════════| 78.2ms
MLFQ     |═══════════| 25.4ms (최고)
WFQ      |═══════════════════════════════| 65.8ms
```

---

4. 알고리즘별 상세 분석

4.1 FCFS 분석

강점: - 가장 단순하고 예측 가능 - 기아 현상 없음 - 오버헤드 최소

약점: - Convoy Effect 발생 (긴 요청이 짧은 요청 블로킹) - 우선순위 처리 불가 - 응답성 부족

적합한 사용 사례: - 요청 간 처리 시간이 비슷한 경우 - 우선순위가 필요 없는 배치 작업 - 성능 비교를 위한 베이스라인

4.2 Priority 분석

강점: - 중요한 요청 즉시 처리 - 명확한 우선순위 차등

약점: - 낮은 우선순위 기아 문제 - 에이징 메커니즘 필요 - 우선순위 결정 정책 필요

적합한 사용 사례: - VIP 사용자 우선 처리 - SLA 기반 서비스 - 긴급 요청이 있는 시스템

성능 특성:

```
URGENT 요청: 평균 12.5ms (FCFS 대비 -80%)
HIGH 요청:   평균 35.2ms (FCFS 대비 -48%)
NORMAL 요청: 평균 85.3ms (FCFS 대비 +18%)
LOW 요청:    평균 180.5ms (FCFS 대비 +138%)
```

4.3 MLFQ 분석

강점: - 짧은 요청 최적화 - 적응형 스케줄링 - 처리량 향상

약점: - 구현 복잡도 높음 - 파라미터 튜닝 필요 - 긴 요청 장등 오버헤드

적합한 사용 사례: - 대화형 서비스 - 혼합 워크로드 - 응답시간이 중요한 시스템

큐 레벨별 성능:

```
Q0 (1초 퀀텀):   평균 25.4ms, 50% 요청 완료
Q1 (3초 퀀텀):   평균 68.3ms, 30% 요청 완료
Q2 (8초 퀀텀):   평균 120.2ms, 15% 요청 완료
Q3 (무제한):     평균 180.2ms, 5% 요청 완료
```

4.4 WFQ 분석

강점: - 가중치 기반 공정성 - 테넌트별 QoS 보장 - 기아 현상 방지

약점: - 가상 시간 계산 오버헤드 - 테넌트 관리 복잡도 - 단일 사용자 환경에서는 불필요

적합한 사용 사례: - 멀티테넌트 SaaS - 차등화된 서비스 등급 - QoS 보장이 필요한 환경

테넌트별 성능:

```
Enterprise (가중치 100): 평균 15.8ms, 대역폭 58%
Premium (가중치 50):     평균 32.5ms, 대역폭 29%
Standard (가중치 10):    평균 85.2ms, 대역폭 6%
Free (가중치 1):         평균 320.5ms, 대역폭 0.6%
```

---

5. 최적화 권장사항

5.1 일반 권장사항

5.1.1 하드웨어

개선 영역	권장 사항	예상 효과
CPU	코어 수 증가	동시 처리량 증가
RAM	8GB 이상	큐 용량 증가
SSD	NVMe SSD	I/O 지연 감소
네트워크	지연 최소화	응답시간 개선

5.1.2 소프트웨어

```
# Node.js 메모리 증가
NODE_OPTIONS="--max-old-space-size=4096"

# Redis 최적화
redis-cli CONFIG SET maxmemory 2gb
redis-cli CONFIG SET maxmemory-policy allkeys-lru

# MongoDB 인덱스 추가
db.requestLogs.createIndex({ requestId: 1 })
db.requestLogs.createIndex({ status: 1, createdAt: -1 })
```

5.2 알고리즘별 권장사항

5.2.1 FCFS 최적화

```
# 동시 처리량 최대화
MAX_CONCURRENT_REQUESTS=10

# 캐싱 활성화
ENABLE_CACHE=true
CACHE_TTL=3600
```

5.2.2 Priority 최적화

```
# 에이징 파라미터 조정
AGING_INTERVAL_MS=30000        # 30초 (기본 60초)
AGING_THRESHOLD_MS=60000       # 1분 (기본 2분)
MAX_AGE_PROMOTIONS=3           # 3단계 (기본 2단계)
```

5.2.3 MLFQ 최적화

```
# 타임 퀀텀 조정 (워크로드에 따라)
MLFQ_TIME_QUANTA=500,2000,5000,0  # 짧은 요청 위주
MLFQ_TIME_QUANTA=2000,5000,10000,0  # 긴 요청 위주

# 부스팅 간격 조정
MLFQ_BOOST_INTERVAL_MS=3000  # 3초 (기본 5초)
```

5.2.4 WFQ 최적화

```
# 가중치 세분화
WFQ_ENTERPRISE_WEIGHT=200  # 더 높은 우선순위
WFQ_PREMIUM_WEIGHT=100
WFQ_STANDARD_WEIGHT=20
WFQ_FREE_WEIGHT=1

# 기본 서비스 시간 추정 조정
WFQ_DEFAULT_SERVICE_TIME=3000  # 3초 (기본 5초)
```

5.3 워크로드별 권장 설정

5.3.1 대화형 서비스 (챗봇)

```
DEFAULT_SCHEDULER=mlfq
MAX_CONCURRENT_REQUESTS=5
MLFQ_TIME_QUANTA=500,1500,4000,0
MLFQ_BOOST_INTERVAL_MS=3000
```

5.3.2 배치 처리 (데이터 분석)

```
DEFAULT_SCHEDULER=fcfs
MAX_CONCURRENT_REQUESTS=10
ENABLE_CACHE=true
CACHE_TTL=7200
```

5.3.3 멀티테넌트 SaaS

```
DEFAULT_SCHEDULER=wfq
MAX_CONCURRENT_REQUESTS=8
WFQ_ENTERPRISE_WEIGHT=100
WFQ_PREMIUM_WEIGHT=50
WFQ_STANDARD_WEIGHT=10
WFQ_FREE_WEIGHT=1
```

5.3.4 하이브리드 환경

```
# 런타임에 스케줄러 전환
# 평상시: MLFQ
# 피크 시간: Priority (VIP 우선)
# 야간 배치: FCFS
```

5.4 모니터링 권장사항

5.4.1 주요 모니터링 메트릭

메트릭	임계값	액션
평균 대기 시간	> 1초	동시 처리량 증가
대기열 길이	> 100	스케줄러 변경 고려
에러율	> 5%	로그 분석
메모리 사용량	> 80%	캐시 정리

5.4.2 알림 설정

```
const alertConfig = {
  waitTimeThreshold: 1000,   // 1초 이상 대기 시 알림
  queueLengthThreshold: 100, // 100개 이상 대기 시 알림
  errorRateThreshold: 0.05,  // 5% 이상 에러 시 알림
};
```

부록: 벤치마크 실행 방법

벤치마크 스크립트 실행

```
# 전체 벤치마크
npm run benchmark

# 특정 시나리오
npm run benchmark -- --scenario uniform
npm run benchmark -- --scenario mixed
npm run benchmark -- --scenario priority
npm run benchmark -- --scenario burst
npm run benchmark -- --scenario multitenant

# 특정 알고리즘
npm run benchmark -- --algorithm fcfs
npm run benchmark -- --algorithm priority
npm run benchmark -- --algorithm mlfq
npm run benchmark -- --algorithm wfq
```

결과 출력

```
# JSON 형식
npm run benchmark -- --output json > results.json

# CSV 형식
npm run benchmark -- --output csv > results.csv

# 그래프 생성
npm run benchmark -- --generate-charts
```

문서 버전: 1.0.0 최종 업데이트: 2026-01-25 작성자: LLM Scheduler 팀